Facultad de ingeniería

Docente

Leidy Mariani Dorado Flórez

Materia

Sistemas Distribuidos

Trabajo

Avance - 2: Documento Arquitectónico

Estudiantes

Heidy Gabriela Jalvin Avirama
Ricardo Adolfo Hoyos
Juan Rodriguez
Jhonn Alexander Pantoja Jimenez

Popayán, Cauca 28 de abril de 2026

Descripción del avance

En el avance actual del proyecto de gestión de citas médicas, ya se han desarrollado e implementado parcialmente varios de los microservicios definidos en la arquitectura. A continuación se describen:

- **Servicio de autenticación:** que permite a los usuarios registrarse e iniciar diligenciando los datos requeridos de forma segura. Además, ya se definió la arquitectura basada en microservicios, donde las diferentes partes del sistema se comunican entre sí mediante solicitudes y eventos.
- **Servicio de citas:**Que es el servicio principal encargado de administrar todo lo relacionado con las citas médicas. Permite a los pacientes consultar la disponibilidad de horarios, agendar nuevas citas, cancelarlas o reprogramarse según sea necesario.
- **API Gateway:** Se ha implementado un API Gateway que actuará como punto central de acceso para los clientes, encargándose de dirigir las solicitudes a los servicios correspondientes.

Funcionalidades Operativas

Servicio de autenticación:
Actualmente, el servicio de autenticación tiene las siguientes funcionalidades operativas:

- **Verificación del servicio:** Permite comprobar que el servicio está en ejecución
- **Prueba de conexión a la base de datos:** Ejecuta una consulta simplemente (SELECT 1) para validar la conexión a la base de datos
- **Registro de usuarios:** permite registrar un nuevo usuario con sus datos que son: nombre, correo, contraseña, teléfono, dirección.
- **Inicio de sesión:** Valida las credenciales del usuario con la base de datos.
- **Lista de usuarios:** Retorna una lista de usuarios con: id del usuario, nombre del usuario, rol del usuario
- **Consulta de disponibilidad:** Este endpoint recibe el id del doctor, realiza una petición HTTP al servicio de citas y retorna los datos obtenidos de ese servicio.

Servicio de citas

Este servicio también está implementado y conectado a su propia base de datos, encargándose de toda la lógica relacionada con las citas médicas.

- **Verificación del servicio:** Confirma que el servicio está activo
- **Prueba de conexión de base de datos:** Valida la conexión con la base de datos mediante una consulta simple.
- **Agendamiento de citas:** Permite crear una cita médica con: id del paciente, id del doctor, fecha, hora, estado.
- **Consulta de citas por paciente:** se recibe el id del paciente como parámetro y retorna:
    1. Lista de pacientes
    2. fecha, hora, doctor, estado.
- **Consulta de disponibilidad:** se consultar todas las citas registradas y se filtra por el id del doctor si se envía el parámetro, este retorna:
    1. Fecha, hora, estado, doctor

Api Gateway

El API Gateway centraliza el acceso a los servicios y expone endpoints unificados

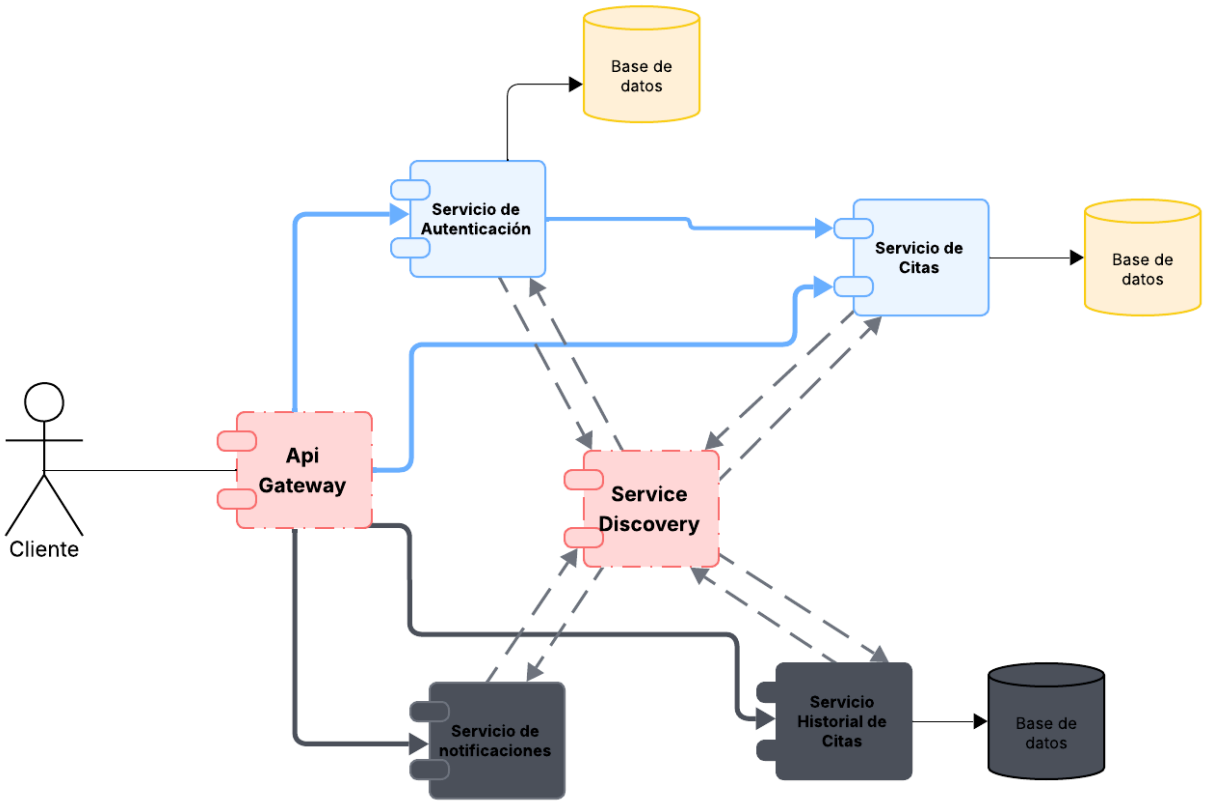
1. **Rutas hacia autenticación**
2. **Rutas hacia gestion de citas**
3. **Manejo de errores**

Componentes que aún no han sido implementados

- servicio de notificaciones
- servicio de historial de citas

Actualmente, aún no se han implementado el servicio de historial de citas, que permitirá almacenar y consultar las citas realizadas, ni el servicio de notificaciones, que se encargará de informar a los usuarios sobre el estado de sus citas.

Diagrama actualizado del sistema.



Comunicación entre servicios

Descripción de la comunicación entre servicios

- Los servicios se comunican mediante HTTP (REST) utilizando la librería requests.
- El servicio de autenticación actúa como cliente, ya que realiza peticiones a otros servicios.
- El servicio de citas actúa como proveedor, ya que expone endpoints con la información de disponibilidad.
- La comunicación se realiza dentro de la red interna de Docker Compose, usando el nombre del servicio como hostname, en este caso:
    1. http://citas:5000

Endpoints utilizados

Servicio de autenticación

1. **GET /**
    - **Función:** Verificar que el servicio esté activo

2. **GET /test-db**
    - **Función:** Probar la conexión con la base de datos de autenticación

3. **POST /login**
    - **Funcion:** Autenticar usuario
    - **Recibe (JSON con correo y contraseña de usuario):**
    - **¿Qué hace?:** Consulta la tabla usuarios y verifica el correo y la contraseña
    - **Respuesta:**
        1. Login exitoso
        2. error 401 si las credenciales son incorrectas.

4. **POST /registrar**
    - **Función:** Registrar nuevo usuario
    - Recibe JSON con:
        1. Nombre del usuario
        2. Contraseña del usuario
        3. correo del usuario
        4. teléfono del usuario
        5. dirección del usuario

5. **GET /listar**
    - **función:** Permite obtener la lista de usuarios registrados

6. **GET / disponibilidad:**
    - Consulta la disponibilidad de citas llamando al servicio citas

Servicios Citas

1. **POST /agendar**
    - **Función**: Crear una cita nueva.
    - Recibe JSON con:
        - id_paciente_citas
        - Id_doctor_citas
        - estado_citas
        - fecha_programacion_citas
        - hora_programacion_citas
    - **Qué hace**: Valida que existan los IDs e inserta la cita en la base de datos.
    - **Respuesta**:
        - Cita creada
        - Error si faltan datos o falla la inserción
2. **GET /disponibilidad**
    - **Función**: Consultar horarios ocupados
    - **¿Qué hace?**: Si recibe id_doctor, filtra por doctor, si no, devuelve todas las citas.
3. **GET / citas pacientes**
    - **Función:** Permite consultar la citas de un paciente por su id
4. **GET /**
    - **Función:** Verificar que el servicio esté activo
5. **GET /test-db**
    - **Función:** Probar la conexión con la base de datos de autenticación

Flujo completo de interacción entre servicios:

Ver la disponibilidad de citas de un doctor

1. El usuario envía una petición al servicio de autenticación:

    - GET /disponibilidad?id_doctor=1

2. Autenticación recibe la solicitud:
    - Recibe el id_doctor, pero no tiene esa información directamente.
    - Entonces decide pedirsela al servicio de citas

3. Autenticación se comunica con Citas:
    - El servicio de autenticación hace una petición interna a: http://citas:5000/disponibilidad?id_doctor=1

4. El servicio de Citas procesa la solicitud:

    - Recibe el id_doctor.
    - Consulta su base de datos.
    - Busca las citas ya registradas para ese doctor.
    - Organiza la información (fechas, horas, estado).

5. Citas responde a Autenticación:
   El servicio de citas devuelve un JSON con los horarios ocupados.

6. Autenticación responde al usuario:

    - Recibe la respuesta de citas.
    - La envuelve en su propio mensaje.
    - Se la envía al usuario

Configuración del entorno

En este sistema, las variables de entorno se utilizan para configurar aspectos importantes sin modificar el código fuente, especialmente datos sensibles y parámetros de conexión.

Las variables de aplicación:

Estas variables son las que usan los servicios en lugar de escribir directamente en el código valores como el host, usuario o contraseña de la base de datos.

Variables de entorno para la conexión a la base de datos del servicio autenticación:

- AUTH_DB_HOST
- AUTH_DB_USER
- AUTH_DB_PASSWORD
- AUTH_DB_NAME

Variables de entorno usadas para la conexión a la base de datos del servicio cita:

- CITA_DB_HOST
- CITA_DB_USER
- CITA_DB_PASSWORD
- CITA_DB_NAME

Estas variables son leídas en el código mediante **os.getenv()**, lo que permite que la aplicación obtenga la configuración en tiempo de ejecución sin mostrar en texto plano las credenciales.

Variables de Infraestructura:

Estas variables son consumidas directamente por los contenedores de la base de datos, en este caso estamos usando MySQL, durante su inicialización.

Servicio autenticación:

- AUTH_MYSQL_ROOT_PASSWORD
- AUTH_MYSQL_DATABASE

Servicio citas:

- CITA_MYSQL_ROOT_PASSWORD
- CITA_MYSQL_DATABASE

Estas variables, son consumidas directamente por el archivo docker compose para configurar los servicios de base de datos. Estas permiten definir la contraseña del superusuario y crea automáticamente la base de datos al levantar el contenedor por primera vez.

Gestión de configuraciones (credenciales, puertos y conexiones)

En el sistema, la gestión de configuraciones se realiza mediante el uso de variables de entorno y Docker Compose, lo que permite separar los datos sensibles y parámetros técnicos del código fuente. De esta manera, se logra una estructura más organizada, segura y fácil de mantener.

Las credenciales de acceso a las bases de datos, como usuarios, contraseñas y nombres de bases de datos, se definen en el archivo .env y son cargadas en cada servicio mediante la propiedad **env_file**. Esto evita que la información sensible esté escrita directamente en el código y permite modificarla fácilmente sin afectar la lógica de la aplicación.

Por otra parte, los puertos de cada servicio se configuran en el archivo docker-compose.yml, donde se establece la relación entre los puertos del contenedor y los de la máquina local.

- Gateway: 5000:5000
- Autenticación: 5001:5000
- Citas: 5002:5000
- Base de datos autenticación: 3307:3306
- Base de datos citas: 3308:3306

El servicio **gateway** actúa como punto de entrada al sistema:

- Recibe las solicitudes del usuario
- Redirige las peticiones a:
    - Autenticación
    - Citas

Portabilidad

La portabilidad del sistema se asegura mediante el uso de contenedores con Docker y la orquestación a través de Docker Compose, lo que permite que la aplicación se ejecute de manera consistente en diferentes entornos sin depender de la configuración específica de una máquina.

Cada servicio, como autenticación, citas y gateway, se ejecuta dentro de su propio contenedor, el cual incluye todas las dependencias necesarias para su funcionamiento. Esto evita problemas relacionados con diferencias en sistemas operativos, versiones de software o configuraciones locales.

Además, el uso de variables de entorno permite adaptar fácilmente el sistema a distintos entornos (desarrollo, pruebas o producción) sin necesidad de modificar el código. Basta con cambiar los valores en el archivo **.env** para ajustar aspectos como credenciales, bases de datos o configuraciones específicas.

Por otra parte, Docker Compose centraliza la definición de todos los servicios, sus puertos, redes y volúmenes, permitiendo levantar todo el sistema con un solo comando. Esto facilita que cualquier persona pueda ejecutar el proyecto en otra máquina con Docker instalado, manteniendo el mismo comportamiento.